

Avaliação Intermediária – IA Generativa (30% da nota final)

Visão Geral

****Objetivo:**** Utilizando uma ferramenta de codificação com IA (Claude Code, OpenAI Codex, GitHub Copilot, Cursor, etc.), desenvolver ****toda a interface e estrutura**** de uma aplicação que resolva um problema real e desafiador – ****sem integrar nenhum modelo de IA/LLM ainda****. A aplicação deve estar funcional como protótipo de UI e acessível por um endpoint público.

****Data de entrega:**** 20/02/2026 (Aula 6)

****Formato:**** Individual

Descrição da Atividade

0 que fazer

1. ****Escolher um problema desafiador**** que se beneficiaria de IA generativa e, de preferência, que seja útil para você após o curso.
2. ****Usar um agente de codificação**** (Claude Code, Codex, GitHub Copilot, Cursor) para desenvolver toda a aplicação: UI, navegação, formulários, visualizações, etc.
3. ****Não integrar nenhum LLM ou modelo de IA**** – onde a IA atuaria, usar respostas simuladas (mock/placeholder). O foco é a estrutura e a interface.
4. ****Publicar um endpoint funcional**** acessível via internet (sugestões: Gradio com share=True, ngrok, Hugging Face Spaces, Streamlit Cloud).
5. ****Criar um repositório GitHub**** com todo o código-fonte.
6. ****Escrever um README detalhado**** documentando o processo, as escolhas de design e a experiência com o agente de codificação. Em particular, explicar o que foi pedido, o que foi implementado corretamente e o que não funcionou.

Exemplos de problemas (do mais desafiador ao menos desafiador)

Nível	Exemplo	Por quê é bom?
-----	-----	-----
Excelente	Sistema de diagnóstico que ajuda técnicos a resolver problemas em máquinas industriais com base em sintomas reportados	Múltiplas telas, fluxo complexo, formulários dinâmicos, histórico
Excelente	Plataforma de análise de currículos que extrai informações e sugere melhorias por seção	Upload de arquivos, parsing, interface de comparação, relatório
Bom	Sistema de triagem médica que coleta sintomas e gera relatório preliminar para o médico	Formulários condicionais, visualização de dados, relatório
Bom	Assistente de documentação técnica que organiza manuais e permite buscas contextuais	Upload, indexação visual, interface de busca, visualização
Fraco	Chatbot genérico de perguntas e respostas	Interface trivial – uma caixa de texto e um botão
Fraco	Tradutor de texto simples	Um campo de entrada, um botão, um campo de saída

> ****Regra geral:**** Se a UI pode ser construída com apenas 1-2 componentes simples, o problema não é desafiador o suficiente.

Critérios de Avaliação

A nota da avaliação intermediária é composta por ****30 pontos**** (equivalentes aos 30% da nota final).

1. Endpoint Funcional (8 pontos)

Pontos	Critério
-----	-----
8	Endpoint acessível, aplicação carrega completamente, todas as telas/abas navegáveis, interações (botões, formulários) funcionam mesmo que com dados simulados
6	Endpoint acessível, aplicação carrega, maioria das funcionalidades opera, pequenos bugs visuais

```
|
| 4 | Endpoint acessível, aplicação carrega mas com funcionalidades parcialmente quebradas |
| 2 | Endpoint acessível mas a aplicação tem erros significativos ou não carrega corretamente |
| 0 | Endpoint não funciona ou não foi compartilhado |
```

2. Complexidade e Ambição do Problema (6 pontos)

```
| Pontos | Critério |
|-----|-----|
| 6 | Problema genuinamente desafiador, com múltiplos fluxos, tipos de interação e componentes de UI variados. Demonstra visão clara de como a IA será integrada futuramente |
| 4 | Problema com complexidade moderada – pelo menos 2-3 telas ou modos de interação diferentes |
| 2 | Problema simples mas com algum esforço de UI além do trivial |
| 0 | Problema trivial (ex: chatbot simples, tradutor básico) |
```

3. Repositório GitHub (4 pontos)

```
| Pontos | Critério |
|-----|-----|
| 4 | Repositório organizado, com commits ao longo do desenvolvimento (não apenas um commit final), estrutura de pastas clara, .gitignore adequado, código legível |
| 3 | Repositório funcional com múltiplos commits, boa organização |
| 2 | Repositório funcional mas com apenas 1-2 commits ou organização básica |
| 1 | Repositório existe mas com código desorganizado ou sem histórico relevante |
| 0 | Repositório não foi compartilhado |
```

4. README – Documentação do Processo (8 pontos)

O README é a parte mais importante da documentação. Deve conter:

```
| Pontos | Critério |
|-----|-----|
| **2** | **Descrição do problema e da solução proposta** – O que o sistema faz? Qual problema resolve? Como a IA será integrada no futuro? |
| **2** | **Escolhas de design** – Por que essa arquitetura? Por que esses componentes de UI? Que alternativas foram consideradas? |
| **2** | **O que funcionou** – Quais partes o agente de codificação gerou bem? Onde a experiência foi positiva? Exemplos específicos de prompts que deram bons resultados |
| **2** | **O que não funcionou** – Onde o agente falhou? O que precisou de intervenção manual? Quais limitações foram encontradas? O que seria feito diferente? |
```

> **Nota:** Honestidade na documentação é valorizada. Um README que diz "tudo funcionou perfeitamente" receberá menos pontos do que um que analisa criticamente a experiência.

5. Uso Efetivo do Agente de Codificação (4 pontos)

```
| Pontos | Critério |
|-----|-----|
| 4 | Evidência clara de uso extensivo do agente (menção de prompts usados, logs ou screenshots, iterações). Demonstra que a maior parte do código foi gerada pelo agente com supervisão do estudante |
| 3 | Uso significativo do agente com alguma documentação da interação |
| 2 | Uso básico do agente, pouca documentação da experiência |
| 1 | Evidência mínima de uso do agente |
| 0 | Sem evidência de uso de agente de codificação |
```

Resumo da Pontuação

```
| Critério | Pontos |
|-----|-----|
| Endpoint funcional | 8 |
| Complexidade e ambição do problema | 6 |
| Repositório GitHub | 4 |
```

```
| README – Documentação do processo | 8 |  
| Uso efetivo do agente de codificação | 4 |  
| **Total** | **30** |
```

Como será a avaliação

No início da Aula 6 (20/02), o professor abrirá rapidamente o endpoint de cada estudante na tela, ao vivo – no máximo **1 minuto por projeto**. O objetivo é apenas verificar que o endpoint está funcional e dar uma olhada rápida no que foi construído. Não haverá apresentação formal nem explicação oral.

> **Importante:** Se por algum motivo você **não** deseja que seu endpoint seja aberto na frente da turma, avise o professor antes da aula. Nesse caso, a verificação será feita de forma privada.

A análise detalhada do repositório e do README será feita pelo professor após a aula.

Entregáveis

1. **Link do endpoint** funcional (Gradio, ngrok, HF Spaces, Streamlit Cloud, ou similar)
2. **Link do repositório GitHub** (público ou com acesso concedido ao professor)
3. **README.md** no repositório com a documentação completa do processo

Arquitetura Recomendada

Para melhores resultados com os agentes de codificação, sugiro a seguinte stack:

- **Backend:** Python (FastAPI ou Flask)
- **Frontend:** React + Vite
- **Banco de dados local:** SQLite (um único arquivo `.db``, sem necessidade de Docker ou servidor de banco de dados – Python já tem suporte nativo via `sqlite3``)

Essa combinação funciona bem porque:

1. Os agentes de codificação geram código Python e React com alta qualidade
2. SQLite não exige instalação nem configuração – é apenas um arquivo
3. FastAPI + React é uma arquitetura moderna que se integra facilmente com APIs de LLM no futuro
4. O ngrok pode expor tanto o backend quanto o frontend com facilidade

> Outras stacks são permitidas (Gradio, Streamlit, Flask puro, etc.), mas a combinação acima tende a gerar os melhores resultados com agentes de codificação.

Dicas

- **Comece pelo design:** Antes de pedir ao agente para codar, planeje as telas e funcionalidades no papel ou num diagrama simples.
- **Itere com o agente:** Não tente gerar tudo em um prompt. Construa incrementalmente – peça uma tela, teste, refine, peça a próxima.
- **Faça commits frequentes:** Isso mostra o progresso e facilita voltar atrás se algo quebrar.
- **Documente enquanto trabalha:** Anote os prompts que funcionaram e os que não funcionaram. Será muito mais fácil escrever o README depois.
- **Mock realista:** Onde a IA atuaria, coloque respostas simuladas que demonstrem como o sistema funcionaria com IA integrada.
- **Teste o endpoint:** Opcionalmente, antes de entregar, peça para um colega acessar o link e verificar se funciona.

FAQ

****P: Posso usar qualquer framework de UI?****

R: Sim. Gradio, Streamlit, Flask, FastAPI + frontend, React, etc. O importante é que funcione e esteja acessível.

****P: Preciso hospedar permanentemente?****

R: O endpoint precisa estar funcional no momento da avaliação. Sugiro soluções temporárias gratuitas como ngrok ou `share=True` do Gradio, desde que estejam acessíveis. Pense como um usuário iria acessar: você não pediria a um cliente para instalar Anaconda e baixar um repositório, certo?

****P: Posso trabalhar em dupla ou grupo?****

R: Não. A avaliação é estritamente individual. Cada estudante deve entregar seu próprio projeto, repositório e README.

****P: E se o agente de codificação gerar código com bugs?****

R: Faz parte! Documente no README: o que o agente errou? Não precisa consertar.

****P: Posso usar código que eu já tinha de outros projetos?****

R: O foco é gerar código novo com o agente. Se reutilizar algo, mencione no README.